Модели данных

С2. Форматы данных. JSON



Московский государственный технический университет
имени Н.Э. Баумана

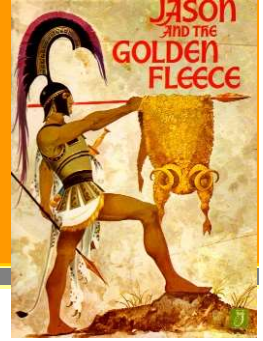
Факультет ИБМ

дек 2024 года

Москва

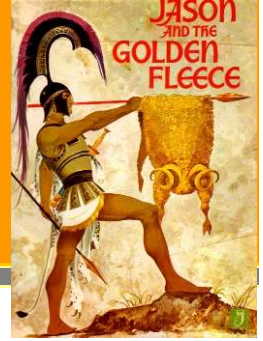
Артемьев Валерий Иванович © 2024

C2. Форматы данных. JSON



- Что такое формат JSON?
- Сценарии применения JSON
- Основные элементы данных JSON
- Именование элементов данных JSON
- Пример JSON-документа
- JSON-схема данных
- Логическая модель JSON-формата
- Табличное описание JSON-формата
- Достоинства и недостатки JSON

Что такое JSON?



JSON (JavaScript Object Notation) — стандартная спецификация универсального текстового формата для хранения и передачи структурированных данных на основе JavaScript.

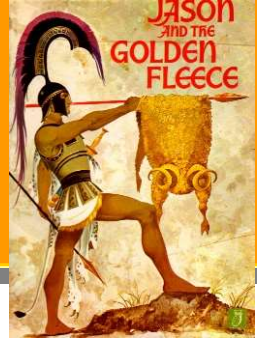
Упрощение структуры данных, более понятный и лёгкий формат. JSON–формат использует ключевые слова, разделители и ограничители для выделения элементов данных. *Кодировка UTF-8*.

Поддерживает *вложенность* и *ограниченный контроль структуры данных*.

Обеспечивает *иерархическое представление данных*.

Стандарты IETF, последний draft 2020-12

Сценарии применения JSON



- *Применение для хранения и обмена данными.*
- *Имеется поддержка типа данных JSON в СУБД.*
- *Поддержка в языках программирования* Ruby, Java, Python.
- *Описание конфигураций.*
- *Создание интерактивных web-страниц с динамическим обновлением контента и JavaScript-приложений* на основе Node.js, AngularJS, React, Backbone, Ember.
- *Разработка мобильного приложения для обмена данными с сервером.*
- *Разработка сложных web-сервисов, взаимодействие с web-сервисами по REST API, сериализация и десериализация данных* (LinkedIn, Twitter, Facebook, Salesforce, GitHub, DropBox, AWS).
- *Тестирование приложений в Junit.*

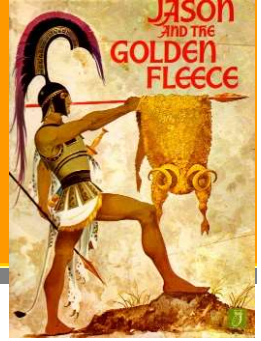
Основные элементы данных JSON



Элементы данных JSON:

- *именованные* элементы данных,
- *неименованные* элементы,
- *простые* элементы (*скаляры*),
- *составные* элементы данных:
  - *объекты* с неупорядоченными элементами,
  - *массивы* с упорядоченными элементами.

Именованные элементы JSON



Именованный элемент – пара "ключ-значение" имеет следующий вид:

"Ключ": значение

Ключ представляет *имя атрибута / свойства*.
Значение может иметь *простой тип данных.*
или являться *составным элементом данных*.

Синтаксис элемента данных:

- *ключ – строка, заключённая в кавычки*
- ключ и значение разделяют *двоеточием.*

Пример:

"city": "Москва"

Синтаксис имён элементов данных



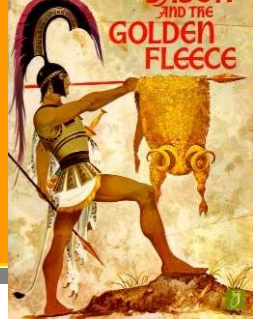
Для имён элементов данных (ключей) используют *способ написания lower camelCase* ("горбатый" стиль) – слияние слов со сменой регистра букв на их стыке, например:

Вид клиента – `видКлиента` (имя элемента)

Альтернативы lower camelCase:

- стиль *upper CamelCase* – с заглавной буквы: `ВидКлиента`,
- стиль *snake_case* ("змейкой") – подчерк для разделения слов: `вид_клиента`.

Составные элементы данных: объекты JSON



Объект содержит *неупорядоченный список именованных элементов данных,* у каждого значение – простой элемент данных или объект или массив.

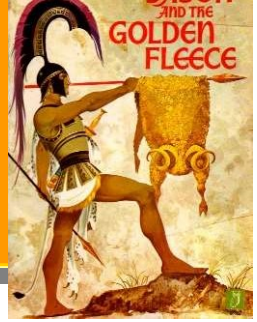
Синтаксис объекта:

- содержимое объекта заключают *в фигурные скобки,*
- пары "ключ - значение" *разделяют запятыми.*

Пример:

```
"Сотрудник":
{"ФИО":"Иванов Пётр Степанович",
"ГодРожд":1987, "ТабельНомер":1234}
```

Составные элементы данных: массивы JSON



Массив содержит *упорядоченный список неименованных элементов данных,* у каждого значение – простой элемент данных или объект или массив.

Синтаксис массивов:

- Массивы *заключают в квадратные скобки*
- Элементы массива *разделяют запятыми*.

Пример:

```
"Авторы":["Чарльз Хоффман","Лив Апнесет Уотсон"]
```

Простые элементы данных JSON (1)



Типы простых элементов данных:

- **числа** (*целые, вещественные с фиксированной и с плавающей точкой*):

```
"количество": 123,
"температура": -21.5,
"числоАвогадро": 6.023E23
```

- **логические значения** (*true - истина, false - ложь*):

```
"isActive": true,
"validated": false
```

- **отсутствующее значение** (*null*)

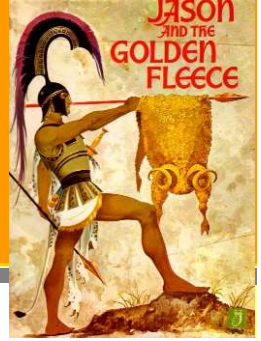
Простые элементы данных JSON (2)



Строки текста:

- **символы** *юникода UTF-8 в двойных кавычках*,
- **спецсимволы** (*escape-последовательности*)
  - \t – табуляция tab,
  - \n – новая строка NL,
  - \r – возврат каретки CR,
  - \f – form feed,
  - \b – backspace
  - экранирование символов \", \\, \/,
- **шестнадцатеричные коды** в юникоде: \uA76BF.
- **дата** и **время** в виде строк.

Пример JSON-документа




```
{
"Library":[
 "book": { "id":1, "topic":"Форматы XML",
 "authors":["Джеймс Бин"], "title": "XML для
 проектировщиков", "Publisher":"КУДИЦ-ОБРАЗ",
 "city":"Москва", "issueYear":2004, "pages":256 },

 "book": { "id":2, "authors":["Бен Форта"], "title:
 "Регулярные выражения", "Publisher":"Изд. дом \"Вилья"мс\"",
 "city":"Москва", "issueYear":2005, "pages":184 },

 "book": { "id":3, "authors":["Чарльз Хоффман", "Лив Апнесет
 Уотсон"], "title": "XBRL для чайников", "Publisher":"ООО
 \"Альфа-книга\", "city":"Москва", "issueYear":2017,
 "pages":432 }
 ]
 }
```

Пример JSON-схемы данных

```
{
  "$schema": "http://json-schema/org\draft-04/schema#",
  "type": "object",
  "properties": {
      "email": { "type": "string" },
      "firstName": { "type": "string" },
      "lastName": { "type": "string" },
  }
}
```

JSON-документ

```
{
  "email": "larrichard@cratic.com",
  "firstName": "Richard",
  "lastName": "Larson"
}
```



Логическая модель JSON-формата

Библиотека [ ]

Книга { }

№п/п
тема
издВид?
название
аннотация?
издательство
город
издГод
страниц?

авторы [ ]

Библиотека [книга]

книга { №п/п
тема?
издВид?
название
аннотация?
издательство
город
издГод
страниц?
авторы [ ]
}



Табличная спецификация JSON-формата

Имя элемента данных	Признак необязательности	Описание элемента данных	Тип данных JSON

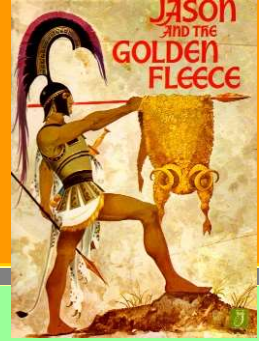
Имена вложенных элементов записываются с отступом

Используются следующие обозначения:
признак необязательности: ?

Типы данных JSON:
объект (object),
массив (array),
число (number),
строка (string),
логическое (boolean)

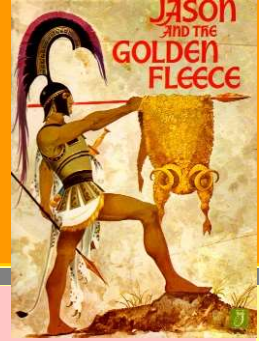


Достоинства JSON-формата



- **Машиночитаемый формат:** текст, понятный людям и машинам, легко читать и писать.
- **Простота:** простой синтаксис и правила вложенности.
- **Гибкость**: поддержка сложных *иерархических структур данных* из объектов, массивов и скаляров.
- **Лёгкость:** объём данных меньше, чем у XML.
- **Универсальность:** *не зависит от прикладных платформ*, может использоваться для различных структурированных типов данных.
- **Расширяемость**: довольно легко расширяется без нарушения существующей структуры данных.
- **Стандартизация**: является международным стандартом де-факто, что обеспечивает его совместимость с различными приложениями
- **Широкая поддержка:** поддержка JavaScript и основными платформами JavaScript, но можно использовать в других языках, наличие инструментов, встроен во многие СУБД в качестве типа.

Недостатки JSON-формата



- **Ограниченный контроль данных**: только проверка синтаксиса и вложенности, контроля типов имеется, но схема данных ещё не стандартизована.
- **Ограниченная поддержка пространства имён:** мешает расширяемости.
- **Не является общим механизмом сериализации и десериализации данных.**
- **Не имеет средств отображения**, подобных CSS2.
- **Нет литералов для даты и времени:** они представлены строками.
- **Нет поддержки комментариев.**
- **Нет поддержки локали:** приходится преобразовывать форматы чисел для отображения.

Вопросы для самопроверки

1. Что такое формат JSON?
2. Сценарии применения JSON
3. Основные элементы данных JSON
4. Именование элементов данных JSON
5. Пример JSON-документа
6. JSON-схема данных
7. Логическая модель JSON-формата
8. Табличное описание JSON-формата
9. Достоинства формата JSON
10. Недостатки формата JSON



Терпения и удачи всем, кто связан с форматами данных

Спасибо за внимание!

Валерий Иванович Артемьев

МГТУ имени Н.Э. Баумана, кафедра ИУ-5

Тел.: +7(985) 365-79-86
e-mail: viart@bmstu.ru